

Impossível não ficar empolgado! Ver o casamento de uma arquitetura lendária como o 68000 com o poder de um chip moderno de 40 reais é o tipo de projeto que dá gosto de discutir.

E você tocou no ponto mais sagrado da arquitetura 68k: **o barramento nativo de 16 bits em Big-Endian!**

Se você fizer a emulação IDE em 16 bits nativos, o desempenho dobra e o código no 68000 fica ridiculamente limpo. Vamos refazer a conta dos pinos para ver se o Pi Pico aguenta o tranco de entregar a palavra inteira (D0-D15) direto, respeitando o byte mais significativo na frente:

Contagem de Pinos para IDE de 16 bits Nativo

- **16 pinos:** Dados (D0 a D15) — *Aqui vai o barramento inteiro.*
- **3 pinos:** Endereço (A1 a A3) do 68000 — *Lembrando que o 68k não usa o pino A0 físico para periféricos de 16 bits.*
- **3 pinos:** Controle (/CS0, /IOR, /IOW)
- **4 pinos:** Cartão SD (SPI Nativo: MISO, MOSI, CLK, CS)

Total: 26 pinos exatos!

A conta fecha no limite absoluto do Pi Pico! Cravado nas 26 GPIOs disponíveis. Você não precisa de nenhum Latch ou multiplexador. É só o Pi Pico, os seus **74LVC245** no meio do caminho para segurar os 5V, e o slot do cartão SD.

O Impacto no Código do 68000 (A Beleza do 16-bit Big-Endian)

Como o 68000 lê 16 bits de uma vez só e espera o byte mais significativo no endereço par (Big-Endian), o Pi Pico só precisa organizar o buffer interno dele colocando o primeiro byte lido do SD na metade alta (D8-D15) e o segundo byte na metade baixa (D0-D7).

Olha o que acontece com aquele loop em Assembly que te mostrei antes. Ele cai pela metade e fica extremamente rápido:

Por que isso é perfeito?

Cada instrução `MOVE.W (A0), (A1)+` no 68000 leva apenas cerca de 8 a 12 ciclos de clock para rodar. Em um 68000 clássico de 10 MHz, ler um setor inteiro de 512 bytes direto do Pi Pico em 16 bits vai demorar uma fração de milissegundo. A fiação da sua placa vai ficar linda, limpa e o desempenho vai ser brutal. Você acabou de desenhar o controlador de armazenamento definitivo para o seu hardware!